Podstawy Uczenia Maszynowego

Metody oparte o sąsiedztwo

Metody oparte o sąsiedztwo

- **neighborhood-based methods**
- zbiór metod wykorzystujących **najbliższych sąsiadów (nearest neighbors)** dla danego punktu
- **idea:** punkty, które są blisko w przestrzeni, powinny mieć podobne własności
- przestrzeń powinna być **semantycznie** sensowna

Zastosowania

- **uczenie nadzorowane,** klasyfikacja / regresja na podstawie podobnych punktów
- **wyszukiwanie,** np. semantic search (Google), visual search (Pinterest), molecular similarity search (Bayer, Pfizer)
- **systemy rekomendacyjne,** np. neighborhood-based collaborative filtering
- **element innych algorytmów:** selekcja cech, redukcja wymiaru, klasteryzacja, …
- **deduplikacja zbiorów danych**

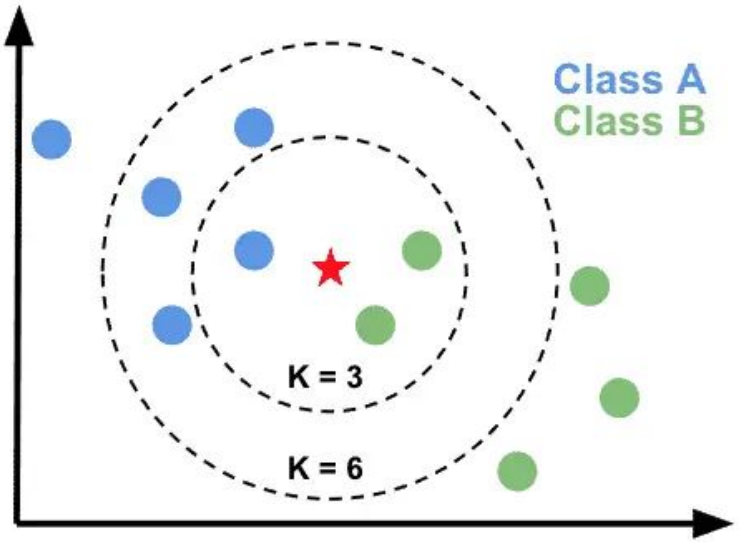
- często dane to **wektory (embeddings)** z sieci neuronowych, np. dla tekstów, obrazów

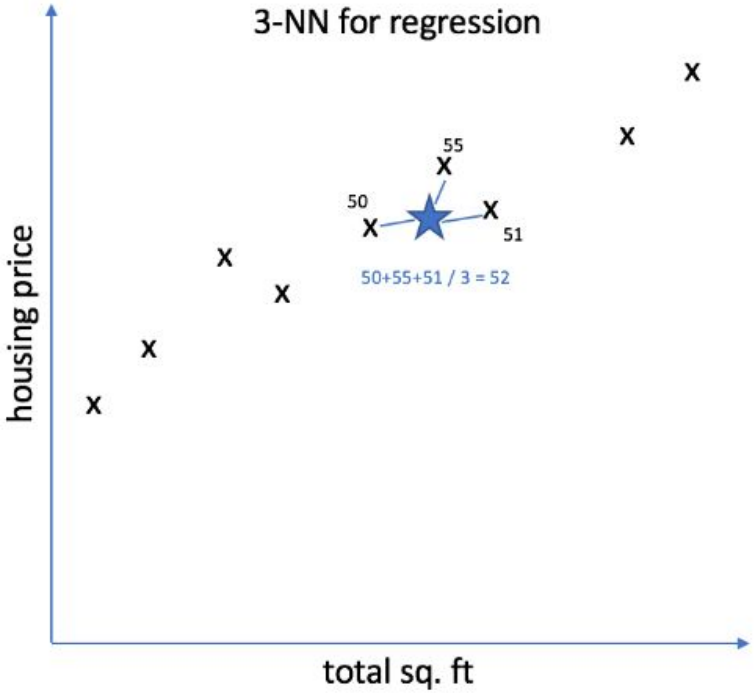
Algorytm kNN

k najbliższych sąsiadów (kNN)

- jeden z najprostszych algorytmów klasyfikacji
- **klasyfikacja:**
  - dla nowego punktu znajdź k najbliższych w zbiorze treningowym
  - zlicz klasy najbliższych sąsiadów
  - zwróć najbardziej popularną klasę
- **prawdopodobieństwo:** zwracamy znormalizowany rozkład klas wśród sąsiadów
- **regresja:** zwracamy średnią wartość zmiennej zależnej wśród sąsiadów

kNN - wizualizacja



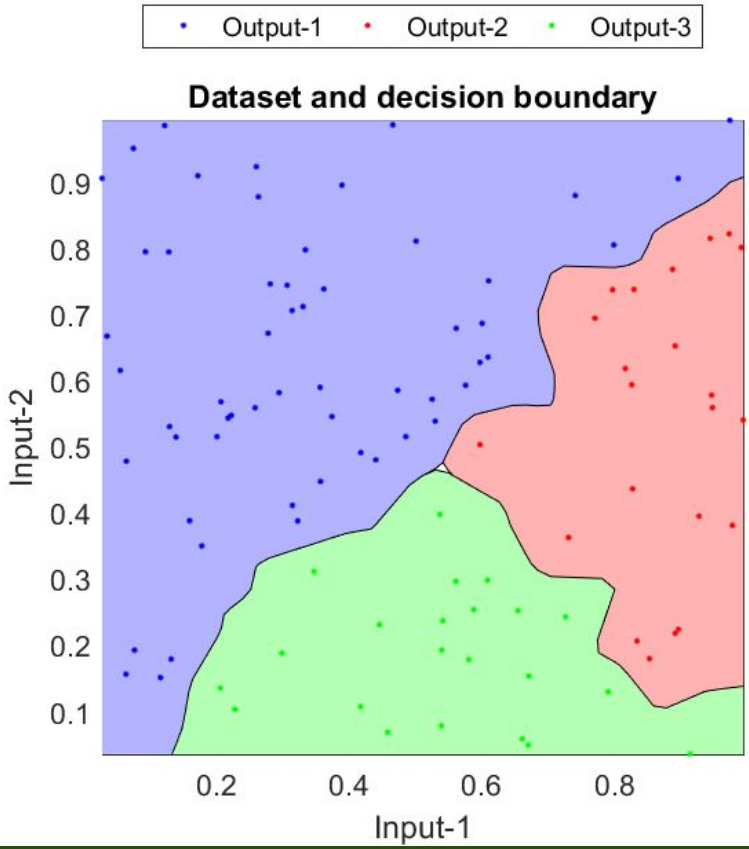


kNN - klasyfikacja i regresja

- **klasyfikacja:**
  - dowolna liczba klas
  - predykcje prawdopodobieństwa
- **regresja:**
  - tylko **interpolacja** - nie potrafi przewidzieć wartości spoza zbioru treningowego

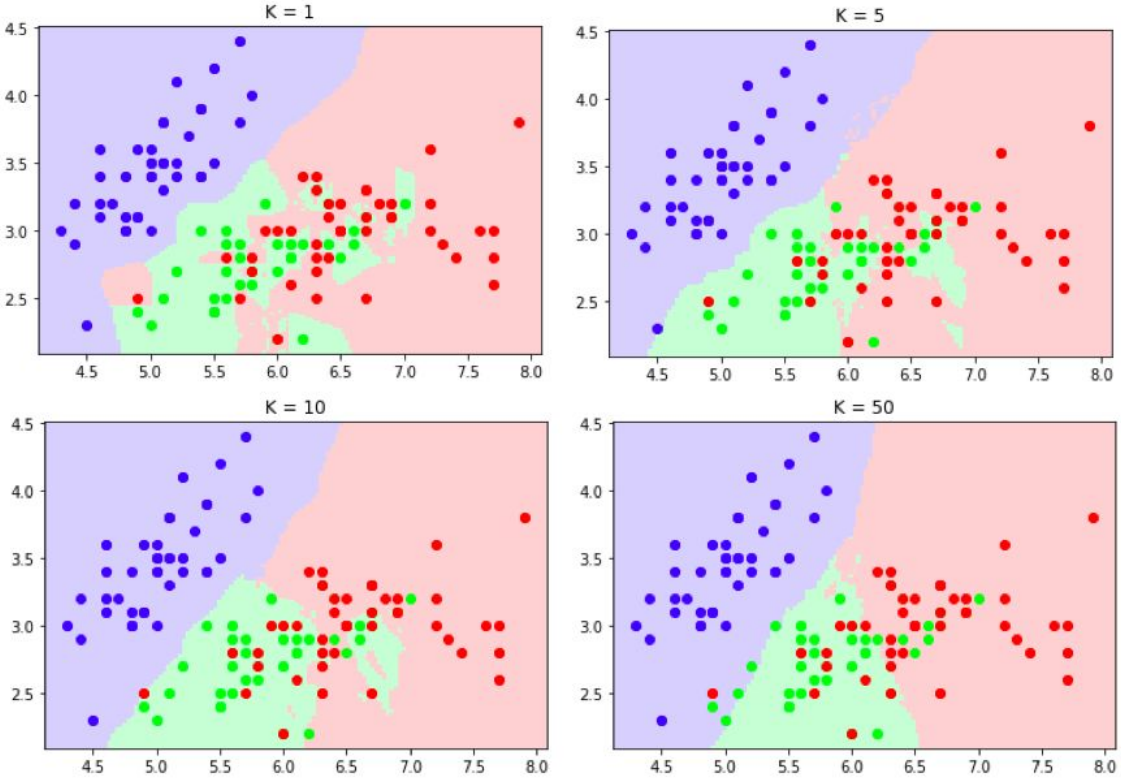
kNN jako model nieliniowy

- kNN ma **nieliniową** granicę decyzyjną
- **model nieliniowy**
- w porównaniu do modeli liniowych (np. regresja logistyczna) ma **dużą pojemność**
- jakość estymacji zależy od **lokalnej gęstości (local density)** punktów



Liczba sąsiadów k

- **bias-variance tradeoff**
- **niskie k:** duża wariancja, "szarpana" granica decyzyjna
- **wysokie k:** duży bias, "gładka" granica decyzyjna
- wpływa na lokalność modelu
- **ogólnie:** low bias, high variance
- **typowe wartości:** od 1 do 50



kNN - hiperparametry

- **liczba sąsiadów k** - jak duże otoczenie uwzględniamy
- **metryka odległości** - co znaczy "najbliżsi" sąsiedzi?
- **ważenie sąsiadów** - czy wszyscy sąsiedzi powinni być tak samo ważni?

- typowo **duża czułość** na liczbę sąsiadów k
- im więcej wymiarów, tym większa czułość na metrykę

"Trening" kNN

- w klasycznym kNN **cała praca** dzieje się na etapie **predykcji (prediction / inference)**
- "trening" = zapisanie zbioru treningowego
- nazywane **lazy learning** albo **instance-based learning**
- **w praktyce:**
  - trening = budowa pomocniczej struktury danych, która przyspieszy wyszukiwanie
  - **search index**

kNN a parametry

- **przypomnienie:** modele liniowe mają *d* parametrów (tyle, co wymiarów)
- **pytanie:** ile jest parametrów w kNN?

kNN jako model nieparametryczny

- **statystyka nieparametryczna (nonparametric statistics)** ma 2 definicje:
  - metody nie zakładające konkretnego rozkładu danych, wnioskujące rozkład prawd. wprost z danych
  - metody bez założenia stałej struktury modelu, dostosowujące uczone parametry oraz ich liczbę do danych
- kNN jest **metodą nieparametryczną**, bo:
  - nie zakłada żadnego rozkładu danych
  - nie ma stałej liczby parametrów - ma ich tyle, ile próbek ma zbiór

Metryki odległości

Metryki

- **metryka (metric)** odległości to drugi najważniejszy hiperparametr
- domyślny wybór to **metryka Euklidesowa (L2)**
- metryki można podzielić na wiele grup, w szczególności:
  - numeryczne
  - dla danych binarnych
  - dla danych mieszanych (numeryczno-kategorycznych)
  - dla tekstów
  - …

Własności metryk

1. Zerowa odległość do samego siebi $d(x, x) = 0$
2. Dodatniość: $x \neq y \Rightarrow d(x, y) > 0$
3. Symetria: $d(x, y) = d(y, x)$
4. Nierówność trójkąta: $d(x, z) \leq d(x, y) + d(y, z)$

1 + 2 => im mniejsza wartość metryki, tym bardziej podobne obiekty.

Kiedy nie jest spełniony ostatni warunek, mamy **półmetrykę (semimetric).**

Metryki a normalizacja danych

- dla danych **numerycznych**
- **praktycznie każda** metryka zakłada, że dane zostały **przeskalowane** do tego samego zakresu wartości
- **powód:** metryki uwzględniają wartości wszystkich cech tak samo, więc żadna nie może dominować

Podobieństwa vs metryki

- **podobieństwo (similarity)** to odwrotność metryki
- im większe podobieństwo, tym bardziej podobne obiekty
- typowo jest **ograniczone (bounded)**, np. do zakresu [-1, 1]
- każde podobieństwo da się zamienić na metrykę i vice versa

Metryka euklidesowa (L2)

- **typowy wzór:** $d_{L2}(a,b) = \sqrt{(a_1-b_1)^2 + (a_2-b_2)^2 + ... + (a_d-b_d)^2} = \sqrt{\sum_{i=1}^{d}(a_i-b_i)^2}$
- za pomocą **iloczynów skalarnych (dot products):**

$$d_{L2}(a,b) = \sqrt{(a-b)\cdot(a-b)} = \sqrt{a\cdot a - 2a\cdot b + b\cdot b}$$

- często **dużo szybsze:**
  - wektoryzacja
  - optymalizacja dla wektorów rzadkich

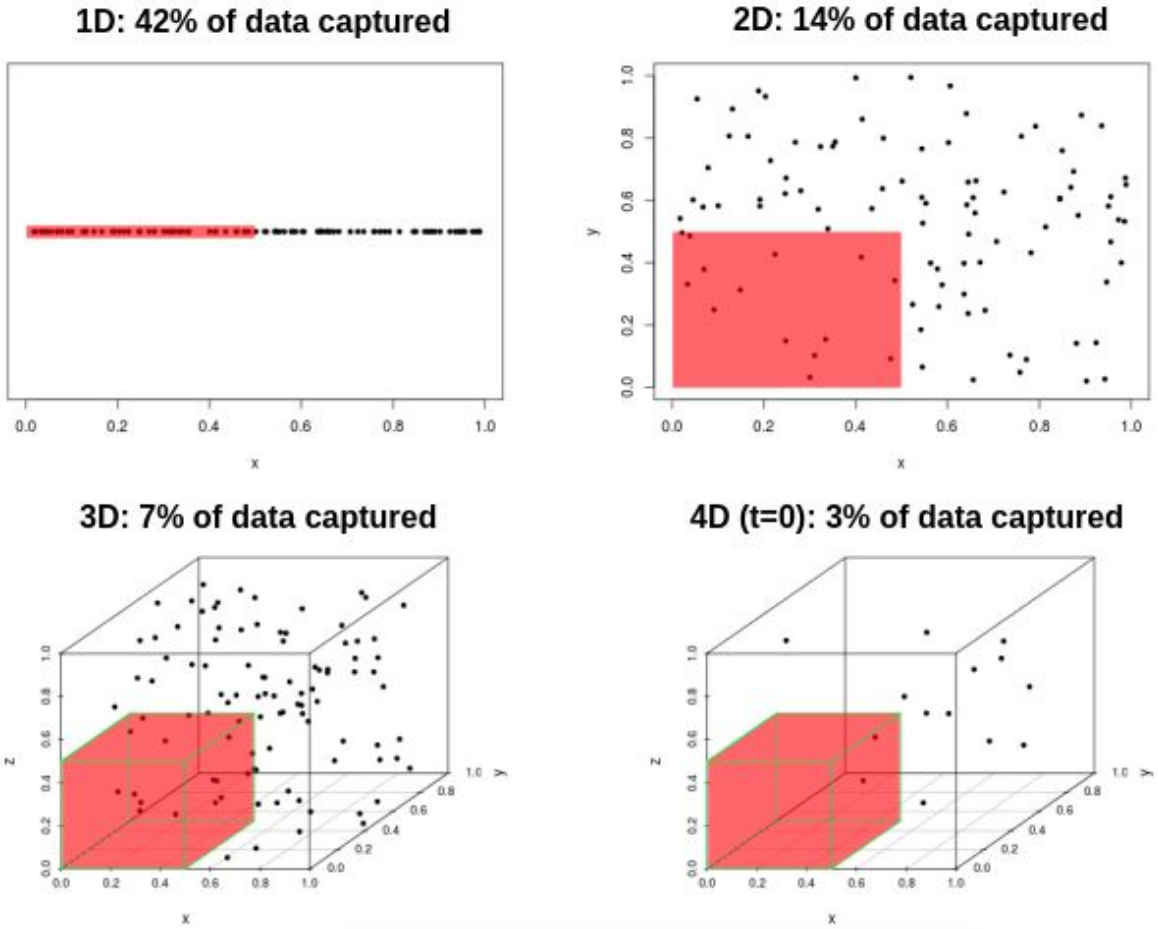
Klątwa wymiaru (curse of dimensionality)

- **obserwacja:** nasze intuicje co do geometrii w niskiej liczbie wymiarów nie działają w przestrzeniach wysokowymiarowych
- **problem 1:** przestrzenie wysokowymiarowe są bardzo puste; wymiar rośnie liniowo, a objętość eksponencjalnie
- **problem 2:** w przestrzeniach wysokowymiarowych odległości między punktami stają się praktycznie identyczne

$$\lim_{d\to\infty} \mathbb{E}\left[\frac{\max_i dist(x_{query}, x_i)}{\min_i dist(x_{query}, x_i)}\right] \to 1$$

- te i inne problemy nazywa się **klątwą wymiaru (curse of dimensionality)**

Klątwa wymiaru (curse of dimensionality)



Klątwa wymiaru i metryki - artykuły

- *"When Is "Nearest Neighbor" Meaningful?"* K. Beyer, J. Goldstein, R. Ramakrishnan, U. Shaft
- *"On the Surprising Behavior of Distance Metrics in High Dimensional Space"* Charu C. Aggarwal, A. Hinneburg, D. A. Keim
- *"On the existence of obstinate results in vector space models"* M. Radovanović, A. Nanopoulos, M. Ivanović

Klątwa wymiaru i metryki - artykuły

Uwaga na artykuł:

"Effects of Distance Measure Choice on KNN Classifier Performance - A Review" H. Alfeilat, A. Hassanat, et al.

- najpopularniejsze porównanie metryk do kNN
- metryki kompletnie niedostosowane do zbiorów danych, brak normalizacji cech - źródło

Klątwa wymiaru - dyskusje

- CrossValidated - Why is Euclidean distance not a good metric in high dimensions?
- CrossValidated - Distance metric and curse of dimensions
- CrossValidated - Euclidean distance is usually not good for sparse data (and more general case)?

Klątwa wymiaru a złożoność obliczeniowa

- **złożoność obliczania odległości d(a, b):** O(d)
- typowo pomija się w artykułach itp., bo jest "oczywiste"
- klątwa wymiarowości może bardzo zwiększyć złożoność obliczeniową!

Disclaimer: złożoności obliczeniowe w dalszej części prezentacji pomijają wymiar, ale w praktycznie każdym miejscu trzeba by pomnożyć przez O(d)

Klątwa wymiaru - co z tym zrobić?

- **użycie innych metryk niż L2** - niektóre są na to mniej czułe
- **redukcja wymiaru, selekcja cech** - mniej wymiarów to mniejszy problem
- **zmienić metodę** - na coś, co nie jest oparte o sąsiedztwo

Metryka Manhattan (L1)

- **wzór:** $d_{L1}(a,b) = ||a-b||_1 = \sum_{i=1}^{d} |a_i - b_i|$
- **zastosowanie:** dane numeryczne, zamiennik L2
- **zalety:**
  - mniej podatne na klątwę wymiaru od L2
  - typowo nieco lepsze wyniki od L2
- zawsze warto wypróbować do klasyfikacji / regresji

Podobieństwo cosinusowe

- **wzór:** $sim_{cos}(a, b) = \cos(\theta_{a,b}) = \frac{a \cdot b}{||a|| * ||b||}$
- **normalizacja do długości 1:** wtedy to po prostu iloczyn skalarny
- **zakres wartości:** [-1, 1] dla liczb rzeczywistych; [0, 1] dla liczb nieujemnych
- **zastosowanie:** wektory rzadkie, np. zliczanie słów w tekstach (NLP)
- **zalety:**
  - szybkie implementacje dla wektorów rzadkich
  - nieco mniej podatne na klątwę wymiaru dla wektorów rzadkich od L2

Metryka L2 a podobieństwo cosinusowe

$$d_{L2}(a,b) = \sqrt{(a-b)\cdot(a-b)} = \sqrt{a\cdot a - 2a\cdot b + b\cdot b}$$

$$sim_{cos}(a,b) = \cos(\theta_{a,b}) = \frac{a\cdot b}{||a|| * ||b||}$$

- jeżeli wektory są jednostkowe:

$$\sqrt{a\cdot a - 2a\cdot b + b\cdot b} = \sqrt{1 - 2cos(\theta_{a,b}) + 1} = \sqrt{2 - 2cos(\theta_{a,b})}$$

Jaccard / Tanimoto similarity

- **oryginalny wzór** służy do porównywania zbiorów:

$$sim_{Jaccard}(a,b) = \frac{|a \cap b|}{|a \cup b|} = \frac{|a \cap b|}{|a| + |b| - |a \cap b|}$$

- w przypadku **wektorów binarnych** wygodniejszy jest zapis:

B \ A	0	1
0	M_{00}	M_{10}
1	M_{01}	M_{11}

$$sim_{Jaccard}(M_{ab}) = \frac{M_{11}}{M_{01} + M_{10} - M_{11}}$$

M_11 - liczba cech, które mają wartość 1 w obu wektorach

M_01 - liczba cech, dla których wektor *a* ma wartość 0, natomiast wektor *b* wartość 1

M_10 - analogicznie

- **zastosowanie:** dane binarne, np. biologia, chemia molekularna, dane ankietowe
- **zalety:** bardzo odporne na klątwę wymiaru, świetne wyniki dla danych binarnych

Inne metryki

- **dane binarne:**
  - Dice, Russel, Simpson, … similarity
- **dane tekstowe:**
  - metryka Hamminga, odległość Levenshteina
- **szeregi czasowe:**
  - dynamic time warping (DTW)
- **molekuły:**
  - Fraggle similarity

Metric learning

- **idea:** nauczyć się metryki dobrej dla konkretnych danych
- w Pythonie: metric-learn, OpenMetricLearning
- **nienadzorowane:**
  - uwzględnić kowariancję (korelację) w danych
  - podobne do redukcji wymiaru, usuwamy szum z danych
  - **Mahalanobis distance**
  - link 1, link 2, link 3, link 4
- **nadzorowane:**
  - "oddalamy" klasy od siebie
  - małe odległości wewnątrz klasy, duże między nimi
  - link 1, link 2, link 3, link 4

Ważenie sąsiadów

Wagi sąsiadów

- **obserwacja:** nie wszyscy sąsiedzi są tak samo ważni, bo mogą być bardzo odlegli
- **idea:** ważenie sąsiadów odległością
- **działanie:**
  - klasyfikacja - sumujemy wagi
  - regresja - średnia ważona
- **często poprawia wyniki:**
  - k jest stałe dla wszystkich punktów
  - lokalna gęstość się różni

Odwrotność odległości

- **wzór:** $w(x_{query}, x_i) = \frac{1}{d(x_{query}, x_i)}$
- jedyna opcja zaimplementowana wprost w Scikit-learn
- **zalety:** często daje lepsze wyniki od braku ważenia
- **wady:** może zbyt mocno skupiać się na najbliższych punktach

Znajdowanie najbliższych sąsiadów

Bazowy kNN i złożoność obliczeniowa

- rozwiązanie typu **"brute force"**, odległości jeden-do-wszystkich
- trening: zapisanie zbioru
- predykcja: znalezienie k-tego najbliższego sąsiada to O(n), powtarzamy k razy
- **złożoność treningu:** O(1)
- **złożoność predykcji:** O(k*n)

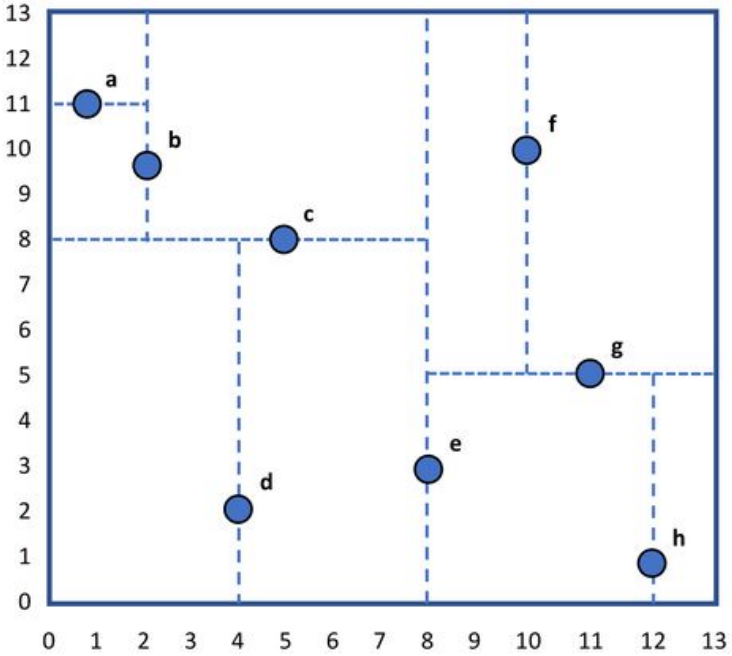
- **lepiej unikać w praktyce**, bo to jest O(k*n) ewaluacji metryki!

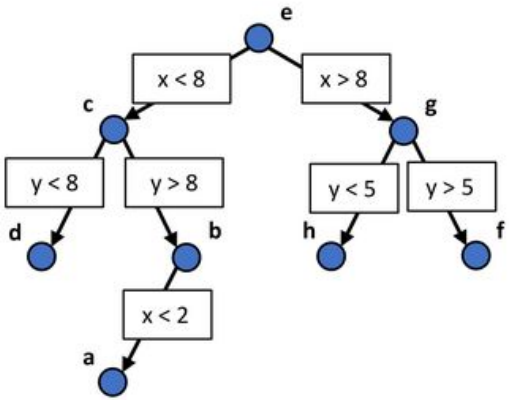
Struktury danych

- **space-partitioning data structures:**
  - pozwalają szybko wyszukiwać bliskie punkty w przestrzeni
  - dzielą przestrzeń na niezachodzące kawałki
  - zastosowania: ML, information retrieval, grafika komputerowa, geografia (GIS)
- w szczególności przyspieszają metody oparte o sąsiedztwo
- **przykłady:** k-d tree, ball tree, quadtree, octree, R-tree
- struktury **dokładne (exact)**
- działają analogicznie do **indeksu bazodanowego (database index)**

k-d tree

- zbalansowane drzewo binarne
- każdy węzeł dzieli przestrzeń **w jednym wymiarze**
- dzieli przestrzeń **prostopadle do osi**

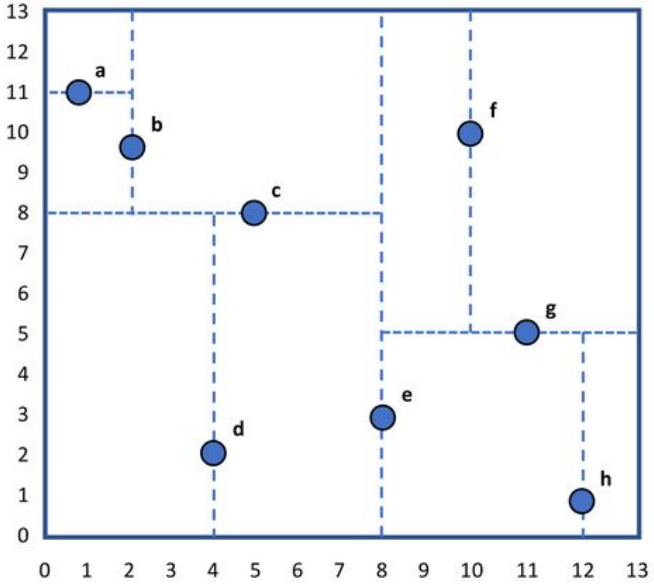


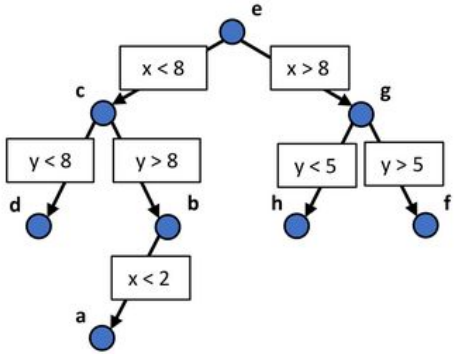


k-d tree - konstrukcja

1. Wybierz wymiar
2. Znajdź medianę
3. Rozdziel punkty według mediany

Powtarzamy rekurencyjnie 1-3 aż do warunku stopu



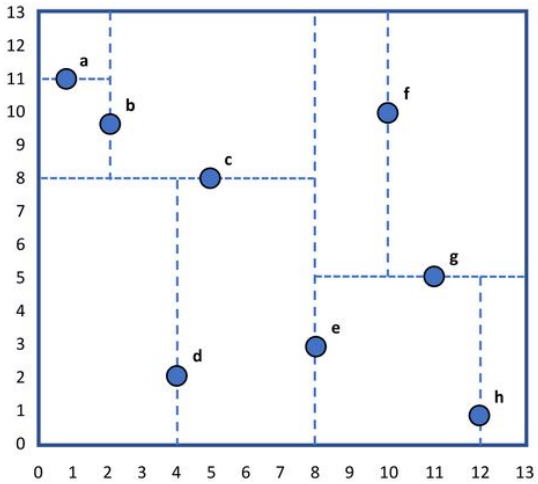


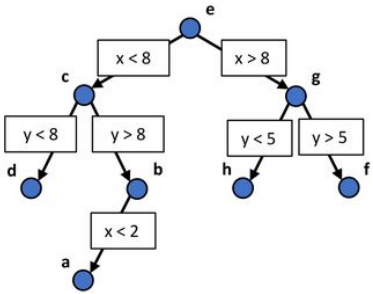
k-d tree - konstrukcja

- **jak wybrać wymiar?**
  - losowo
  - po kolei
  - malejąco według wariancji
- **kiedy przerwać?**
  - tylko 1 przykład w liściu
  - co najwyżej M przykładów w liściu (wtedy brute force)

k-d tree - wyszukiwanie

- schodzimy aż do liścia, jak w zwykłym drzewie binarnym
- w liściu znajdujemy k najbliższych punktów
- cofamy się rekurencyjnie, sprawdzając sąsiednie poddrzewa, jeżeli może tam być lepszy sąsiad
- [prezentacja University of Washington o k-d tree]





k-d tree - złożoność obliczeniowa

- **dodanie punktu:** O(log(n))
- **wyszukanie punktu:** O(log(n))
- **wyszukanie k najbliższych sąsiadów:** około O(k*log(n))

kNN z k-d tree - złożoność obliczeniowa

- trening: konstrukcja k-d tree
- predykcja: wyszukanie k najbliższych sąsiadów w k-d tree
- **złożoność treningu:** O(n*log(n))
- **złożoność predykcji:** O(k*log(n))

  n - liczba punktów w zbiorze treningowym

- **znacznie mniej** ewaluacji metryki!

k-d tree - zalety i wady

Zalety:

- prostota
- szybkość dla małych i średnich zbiorów

Wady:

- czułe na klątwę wymiaru
- typowo wspiera tylko metryki Minkowskiego (euklidesowa, Manhattan, cosinus, …)

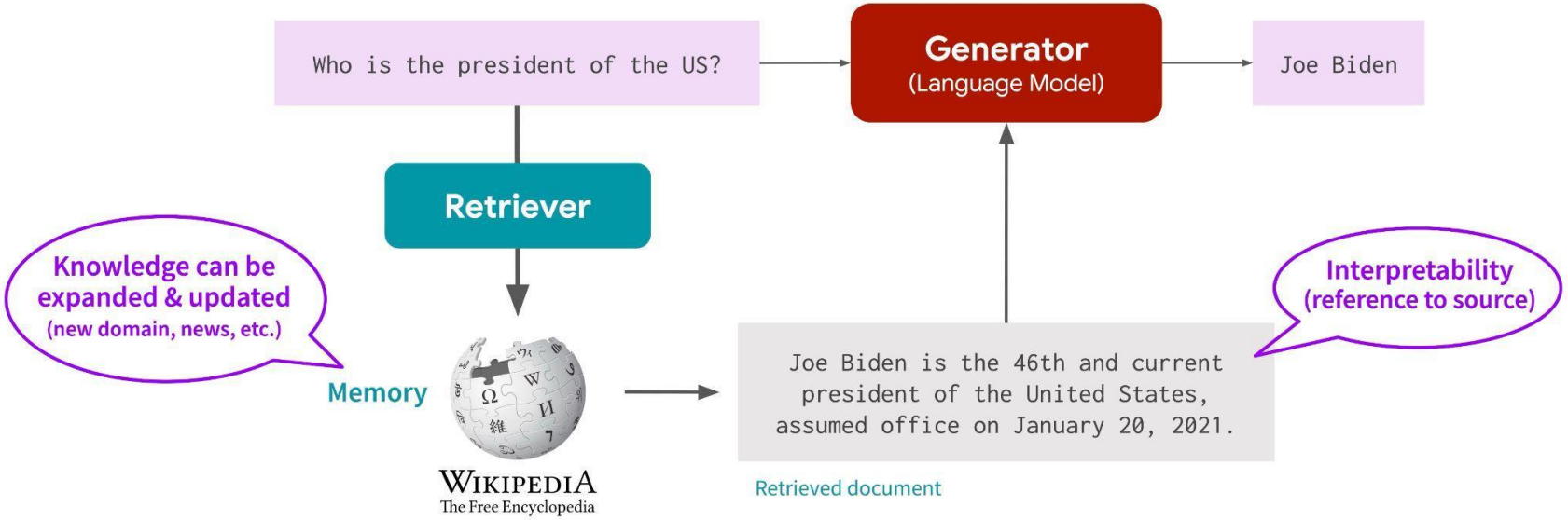
Trzeba jeszcze szybciej

- **problem:** złożoność O(k*log(n)) jest dalej często zbyt duża w praktyce
- szczególnie problematyczne w **wyszukiwaniu**
- np. Amazon oferuje setki milionów produktów

- często wyszukuje się **wektory (embeddings)** z sieci neuronowych
- typowo 512 / 768 / 1024 wymiary
- k-d tree działa słabo

Retrieval Augmented Generation (RAG)

- modele LLM ulepsza się **bazą wiedzy**, dodając podobne teksty z bazy do promptu
- tzw. **Retrieval Augmented Generation (RAG)**
- jest to problem wyszukiwania k najbliższych sąsiadów w embeddingach z tekstów
- bazy wiedzy jest ogromna - często 1 wektor per paragraf tekstu!



Approximate Nearest Neighbors (ANN)

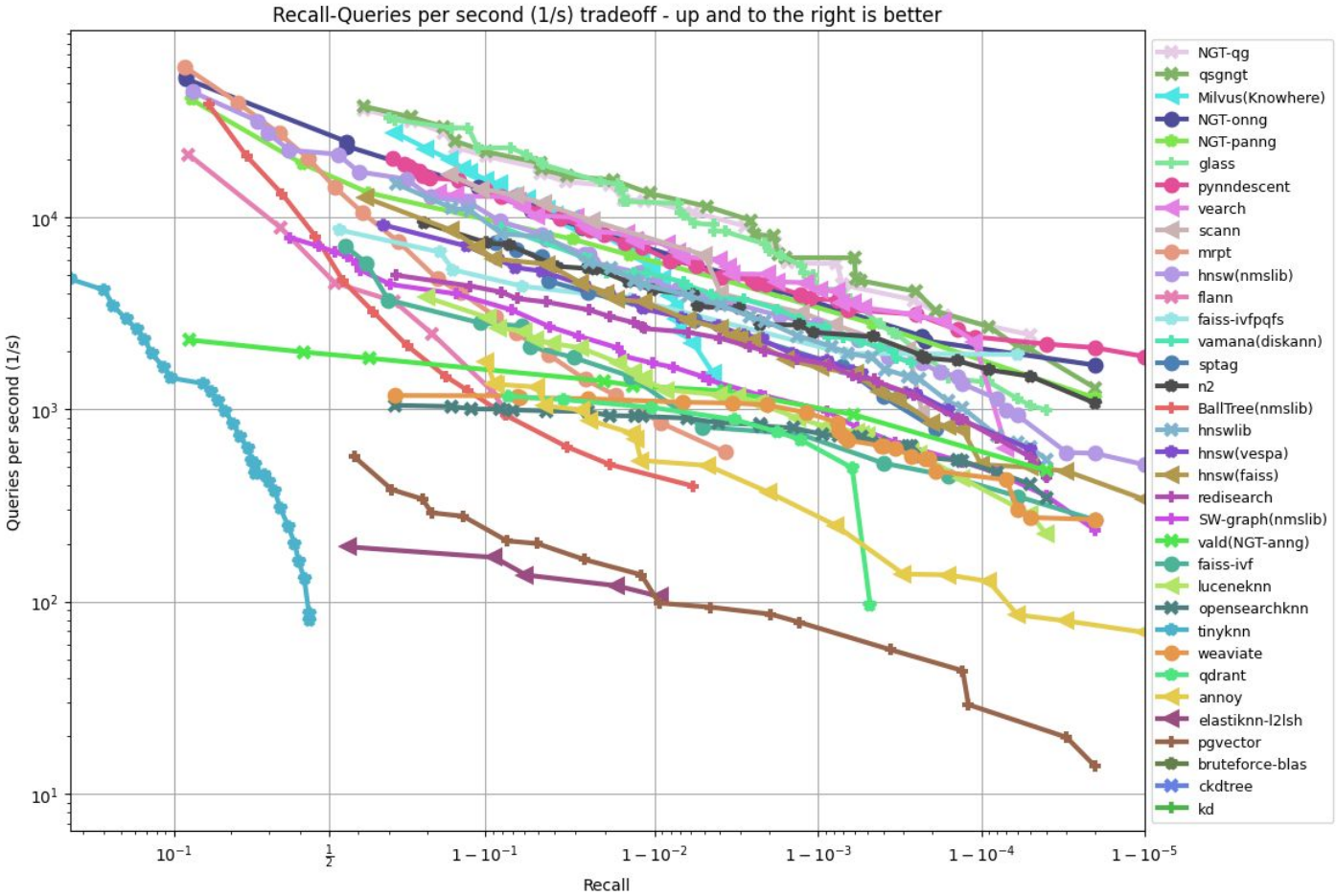
Approximate Nearest Neighbors (ANN)

- **idea:** zamiast rozwiązywać problem dokładnie, znajdźmy rozwiązanie przybliżone
- algorytmy **Approximate Nearest Neighbors (ANN)**
- **speed-recall tradeoff:** będzie szybciej, ale znalezieni "najbliżsi" sąsiedzi mogą nie być najbliżsi
- **czułość (recall):**
  - jak dużo % przykładów pozytywnych jest faktycznie pozytywnych
  - w tym kontekście - ile % faktycznych najbliższych sąsiadów udało się znaleźć
- **naprawdę warto:** 1000x szybciej, recall 99.9%

ANN Benchmark

- link
- świetny benchmark i przegląd bibliotek i algorytmów ANN
- regularnie aktualizowany

ANN Benchmark - Fashion MNIST

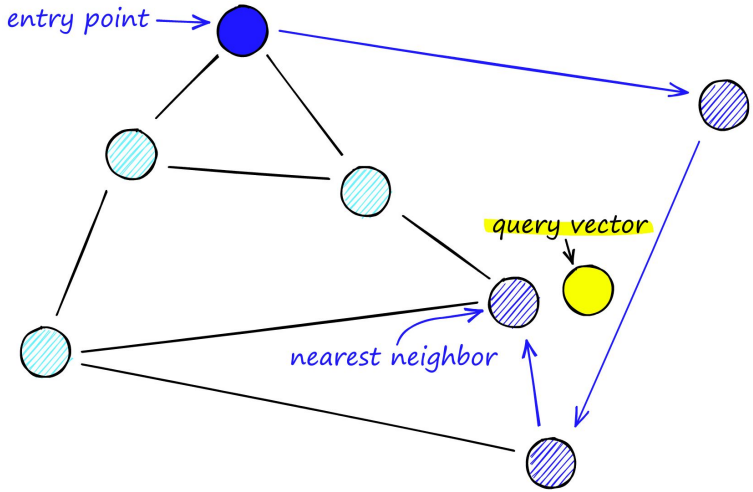


ANN - podejścia

- podejścia różnią się tym, jak znajdują **przybliżoną okolicę** naszego punktu, którą potem dokładniej przeszukują
- **popularne podejścia:**
  - Inverted File Index (IVF)
  - Locality Sensitive Hashing (LSH)
  - Random Projection Forest (RP Forest)
  - algorytmy grafowe na grafach najbliższych sąsiadów (NN graphs)
- **metody grafowe** są obecnie najpopularniejsze, bo mają dobrą jakość i skalowalność

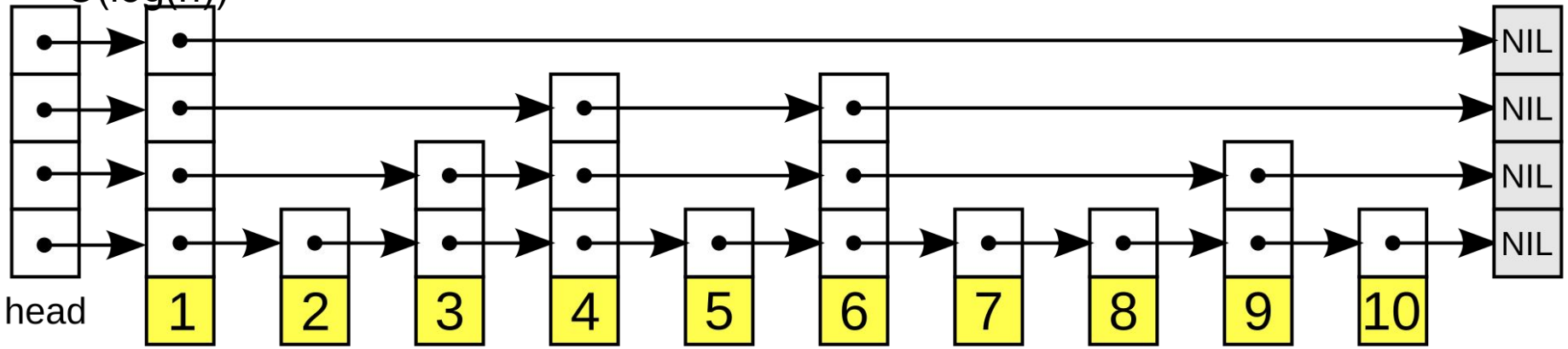
Algorytmy grafowe

- metody używające **grafu najbliższych sąsiadów**
- wierzchołki - punkty ze zbioru
- łączymy krawędziami M najbliższych sąsiadów
- **wyszukiwanie:**
  - typowo zachłanna heurystyka
  - zaczynamy z losowego punktu
  - liczymy odległości sąsiadów i zapytania (query)
  - idziemy do punktu z najmniejszą odległością
- obecne **state-of-the-art (SOTA)** algorytmów ANN



Skip list

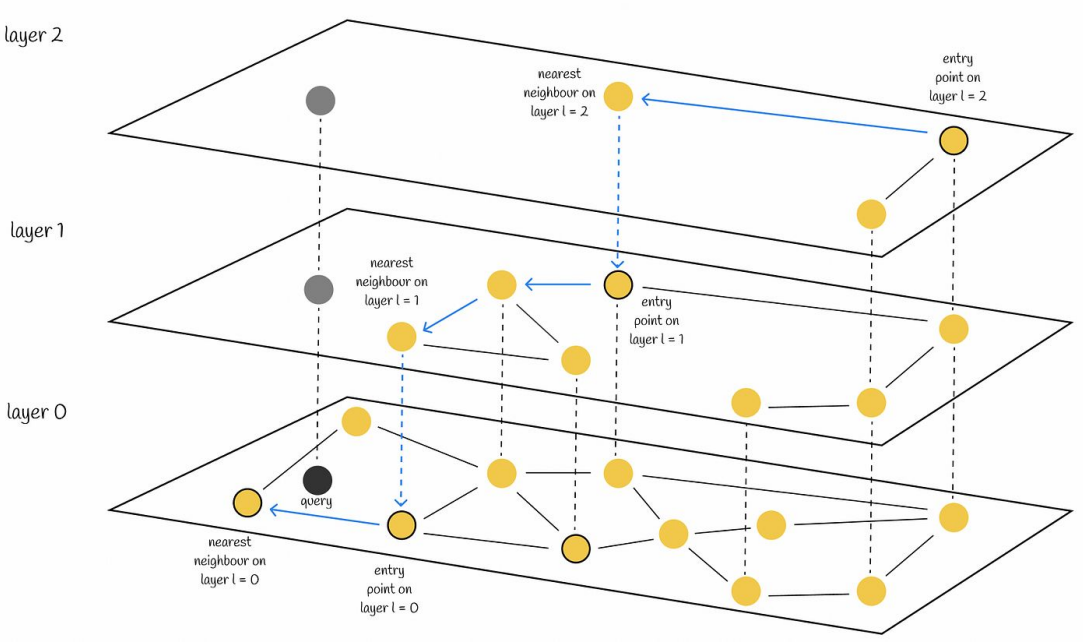
- **probabilistyczna** struktura danych
- posortowana lista + **hierarchia** skip connections do szybkiego skakania
- wstawianie **losuje** (np. 50%), że element ma wskaźnik poziom wyżej, daje to rzadkie i dalekie połączenia
- szukamy od góry, w razie potrzeby schodzimy w dół (gęstsza lista), średnio O(log(n))



Hierarchical Navigable Small World (HNSW)

- **HNSW** to najpopularniejszy obecnie algorytm ANN
- buduje hierarchię grafów - grafowa skip lista
- przeszukujemy od górnych do dolnych warstw, żeby szybko "skakać" po przestrzeni

- górne grafy są rzadkie i szybkie, dolne gęstsze i dokładniejsze
- **zalety:** bardzo dokładne i szybkie
- **wady:** złożoność pamięciowa (indeks trzymany w pamięci), kilka hiperparametrów
- link 1, link 2, link 3, link 4
- oryginalny artykuł



Algorytmy grafowe - inne przykłady

NN-Descent:

- buduje "niedokładny" graf jakąś prostą i szybką metodą, np. RP Forest (patrz slajdy bonusowe)
- potem iteracyjnie go ulepsza
- zbudowany do środowisk rozproszonych
- nie ma jednego globalnego indeksu (ale to zależy od implementacji!)
- link 1, link 2
- oryginalny artykuł

DiskANN (Vamana):

- buduje losowy graf, potem go iteracyjnie ulepsza
- usuwa niektóre niepotrzebne krawędzie
- dodaje dalekie krawędzie, żeby móc "skakać" szybko po grafie
- trzyma indeks na dysku, mocno zoptymalizowane pod to
- link 1, link 2, link 3
- oryginalny artykuł

ANN - dodatkowa literatura

- dobre overview:

W. Li et al. *"Approximate Nearest Neighbor Search on High Dimensional Data - Experiments, Analyses, and Improvement"*

- inne ważne artykuły:

Y. Malkov, D. Yashunin *"Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs"*

S. Subramanya, et al. *"DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node"*

W. Dong et al. *"Efficient K-Nearest Neighbor Graph Construction for Generic Similarity Measures"*

Podsumowanie

Ulepszenia i hiperparametry w kNN

Jakość:

- liczba sąsiadów k
- dobór metryki
- ważenie sąsiadów

Szybkość:

- struktury danych, np. k-d tree
- ANN:
  - IVF
  - LSH
  - Random Projections
  - algorytmy grafowe

Metody oparte o sąsiedztwo - zalety i wady

Zalety:

- prostota
- duża pojemność
- interpretowalność - można zbadać najbliższych sąsiadów
- skalowalność (dla ANN)

Wady:

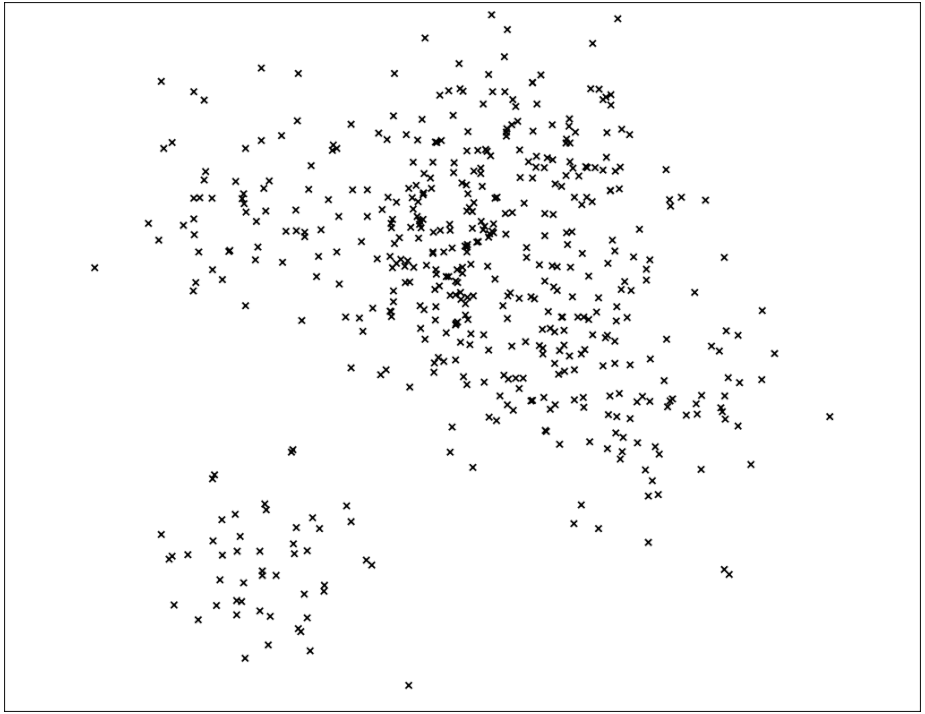
- typowo przeciętne wyniki
- klątwa wymiaru
- złożoność obliczeniowa (dla rozw. dokładnego)
- typowo tylko dane numeryczne

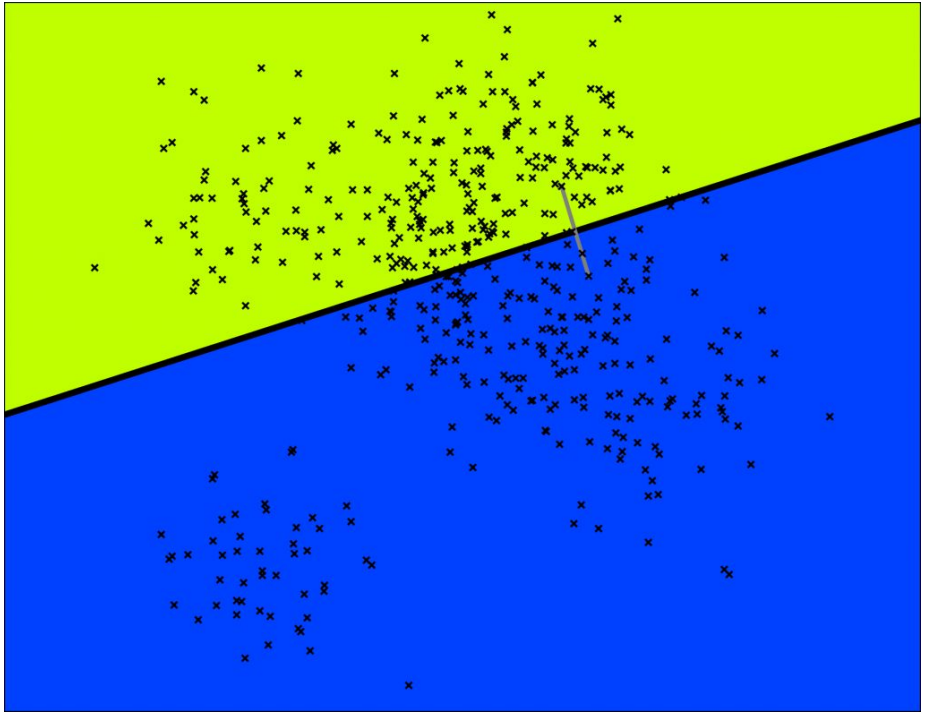
Slajdy bonusowe

Random projection trees (RP trees)

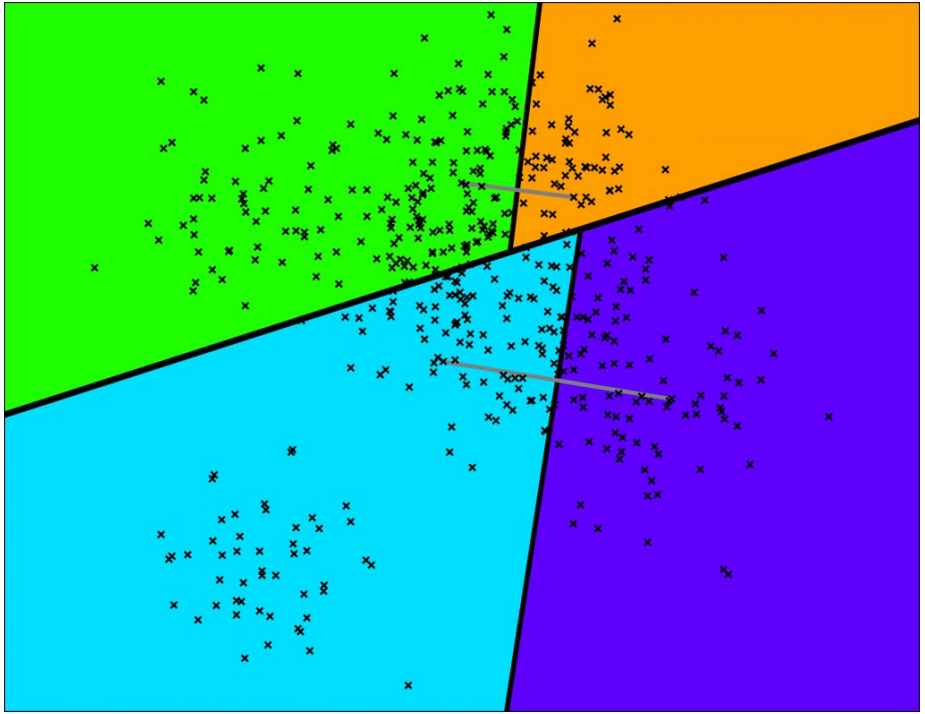
- **idea:** zamiast robić dokładny podział jak w k-d tree, zróbmy go losowo
- **idea 2:** zróbmy dużo podziałów po kolei, tworząc drzewo
- **random projection tree (RP tree):**
  - wybierz losowo 2 punkty ze zbioru
  - na środku między tymi punktami przebiega hiperpłaszczyzna
  - podziel nią punkty na dwie grupy, zrób z nich węzły-dzieci
  - wywołaj rekurencyjnie dla dzieci
  - przerwij, gdy zostanie co najwyżej X punktów w węźle

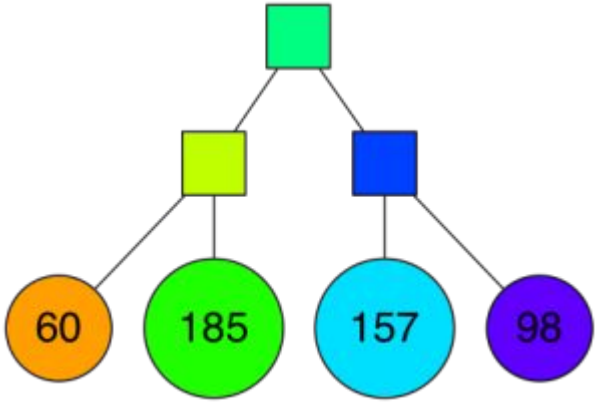
Random projection trees (RP trees)



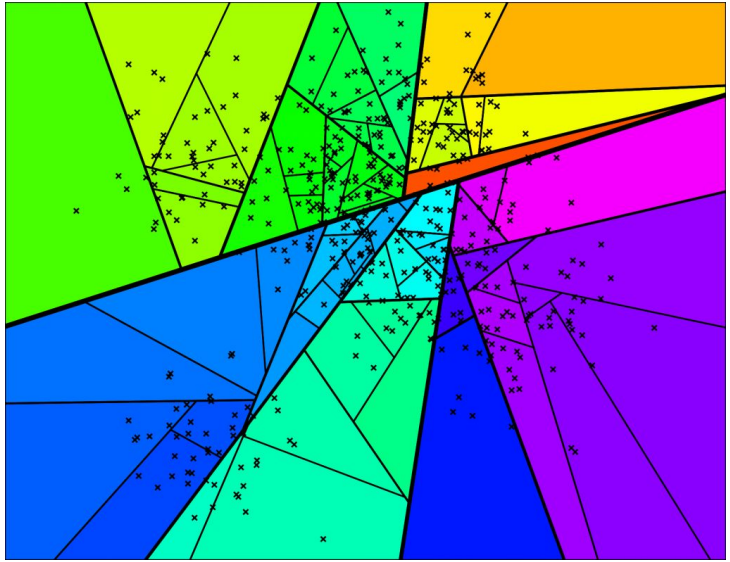


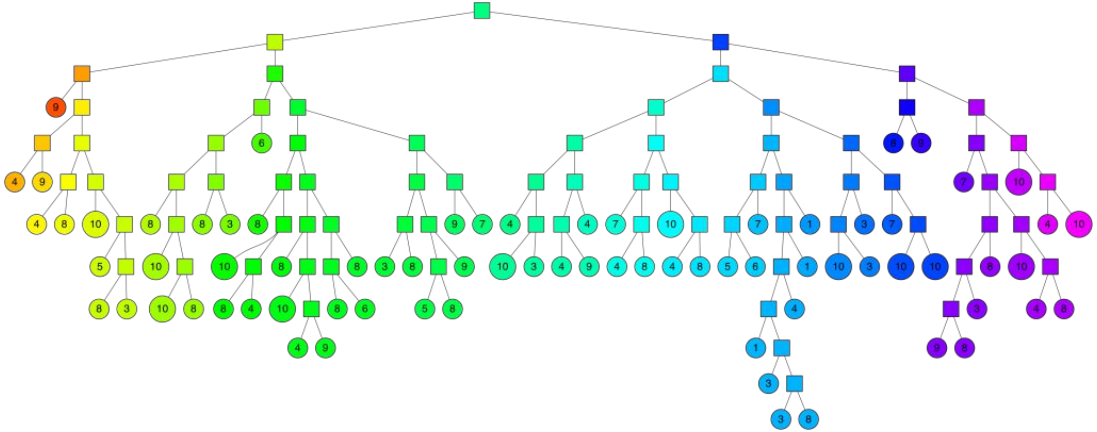
Random projection trees (RP trees)



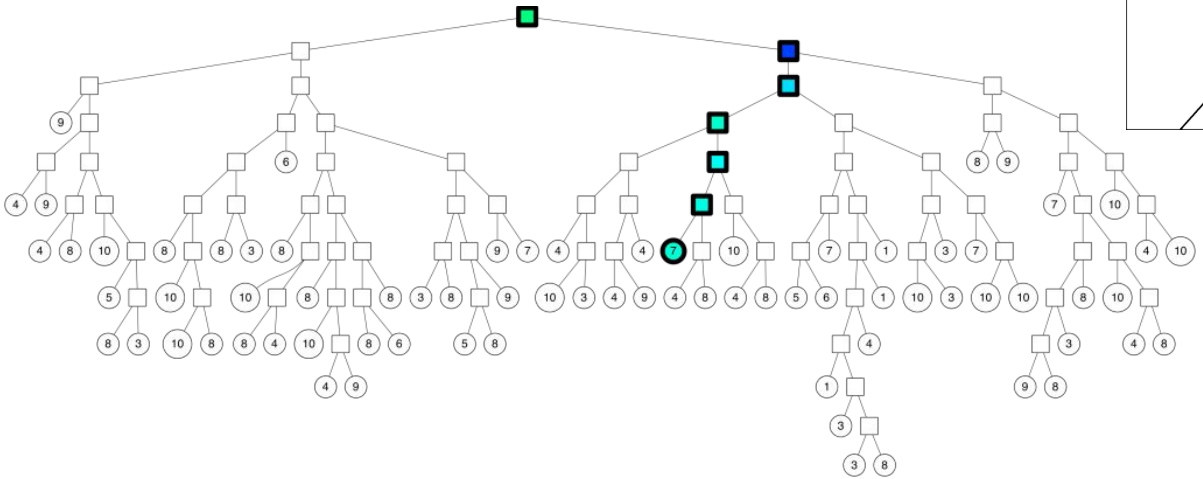


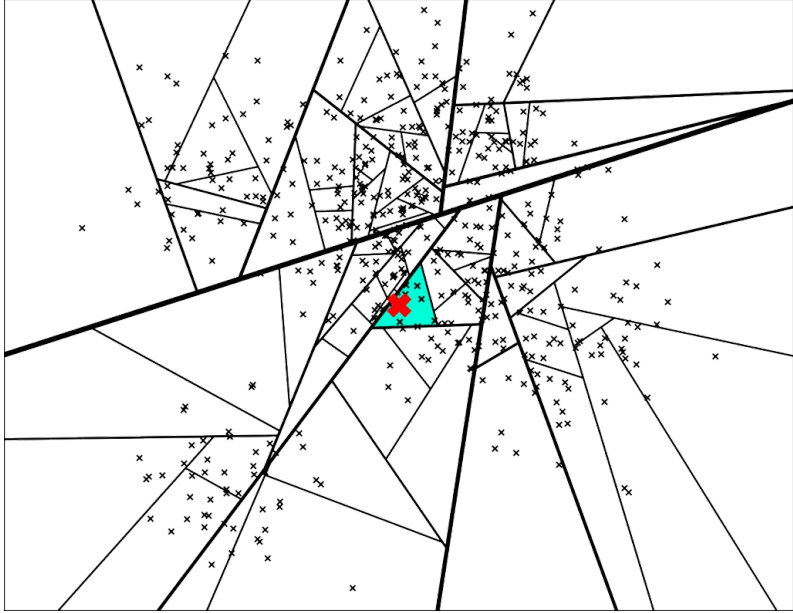
Random projection trees (RP trees)





RP trees - wyszukiwanie

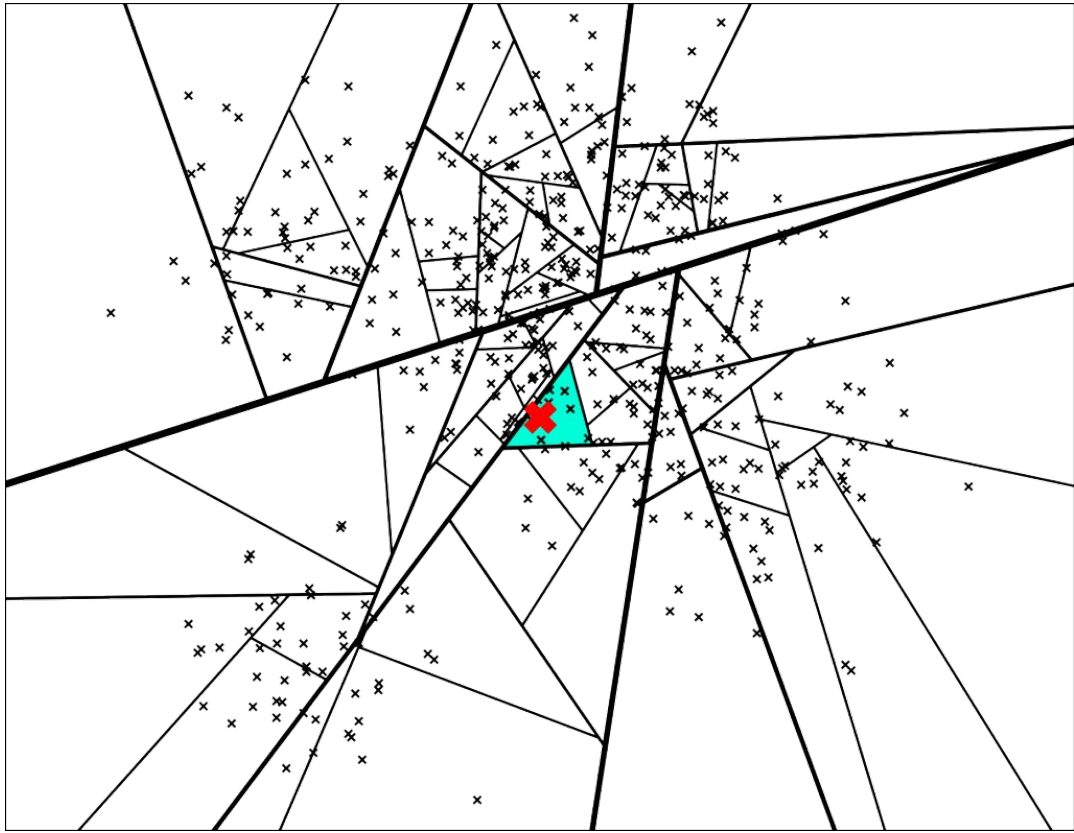




RP trees - problemy

Problem 1: co, jeżeli chcemy więcej sąsiadów, niż jest w finalnym kawałku?

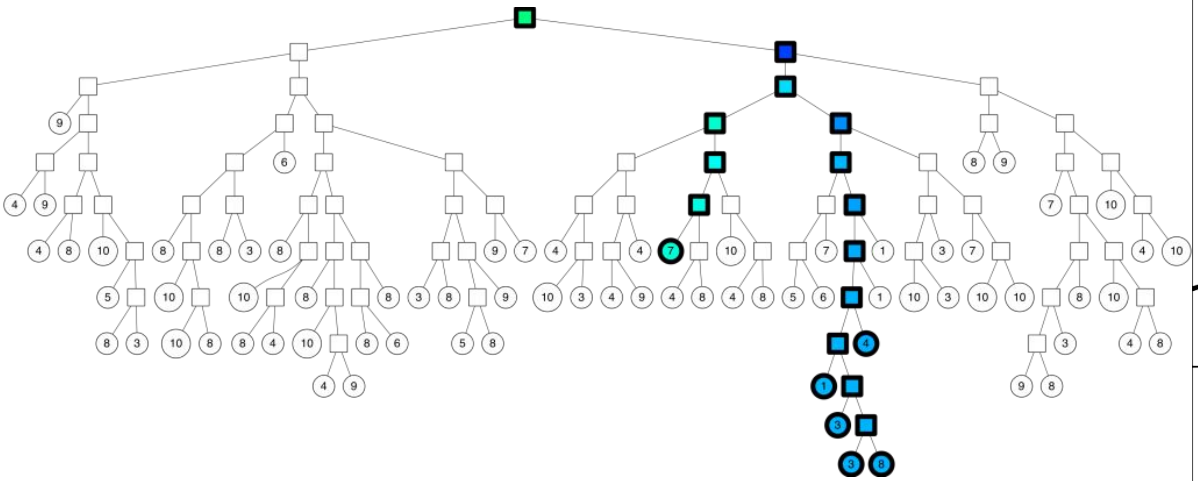
Problem 2: Pominęliśmy oczywistych, bliskich sąsiadów

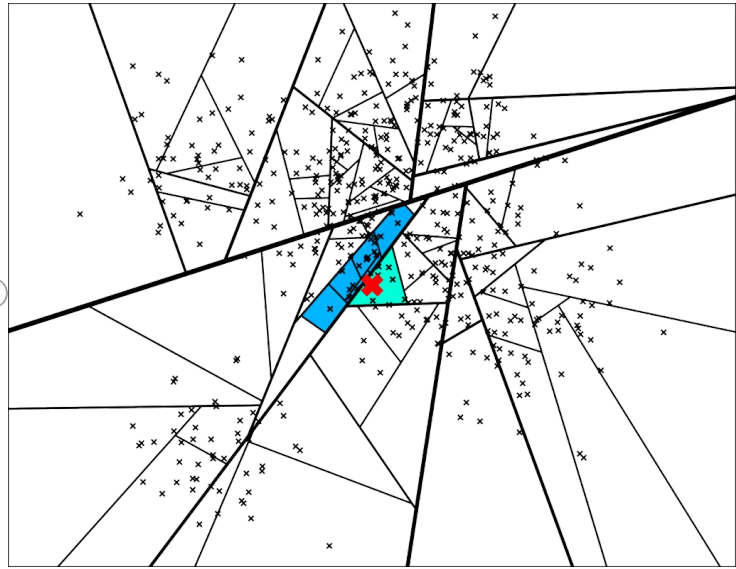


RP trees - priority queue

- **rozwiązanie problemu 1:** sprawdzać wiele poddrzew
- podobne do backtrackingu z k-d tree
- sprawdzamy **najbardziej obiecujące** węzły, pamiętając je w kolejce priorytetowej
- finalnie przeszukujemy kilka "kawałków" przestrzeni

RP trees - priority queue

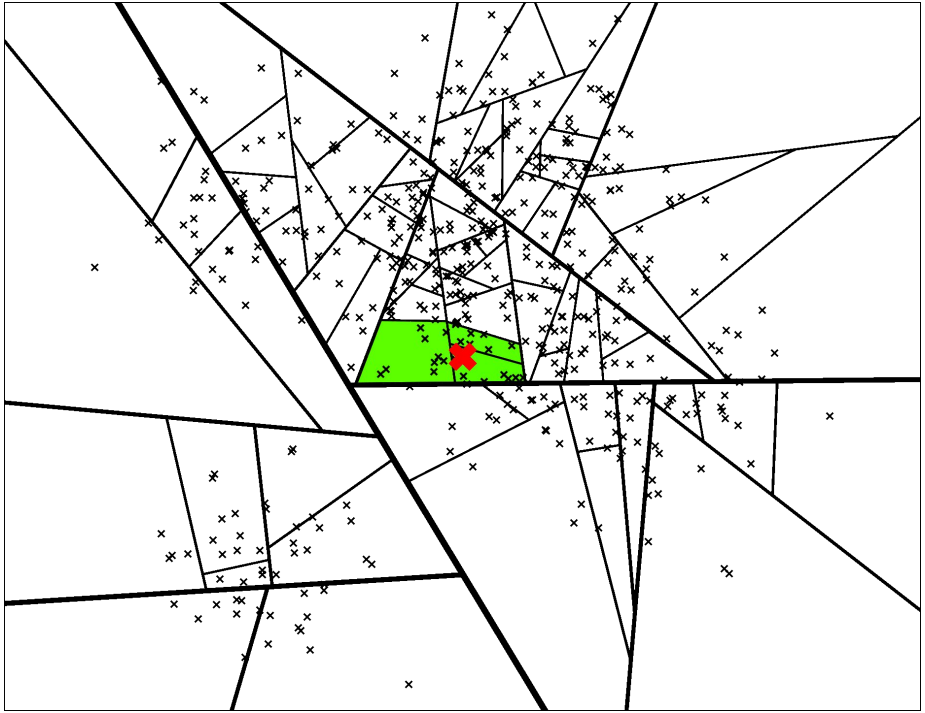


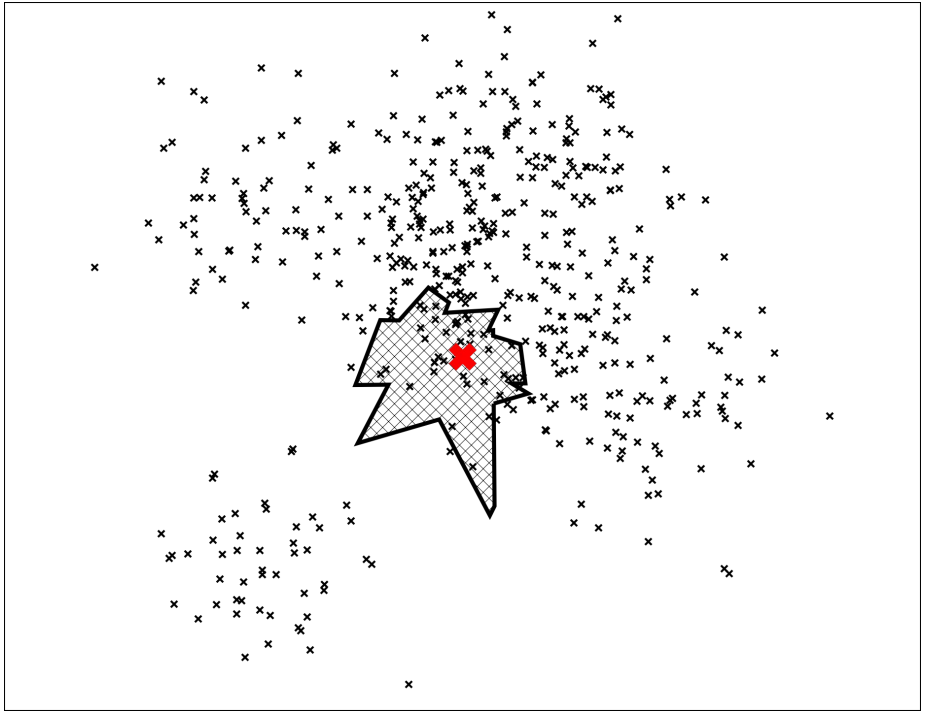


RP forest

- **rozwiązanie problemu 2:** użyjmy wielu drzew naraz
- skoro drzewo jest losowe, to wylosujmy dużo drzew i chodźmy po wszystkich naraz
- można mieć **jedną kolejkę** priorytetową dla wszystkich drzew
- **łączymy obszary** z liści ze wszystkich drzew
- trzeba dokonać **deduplikacji** punktów

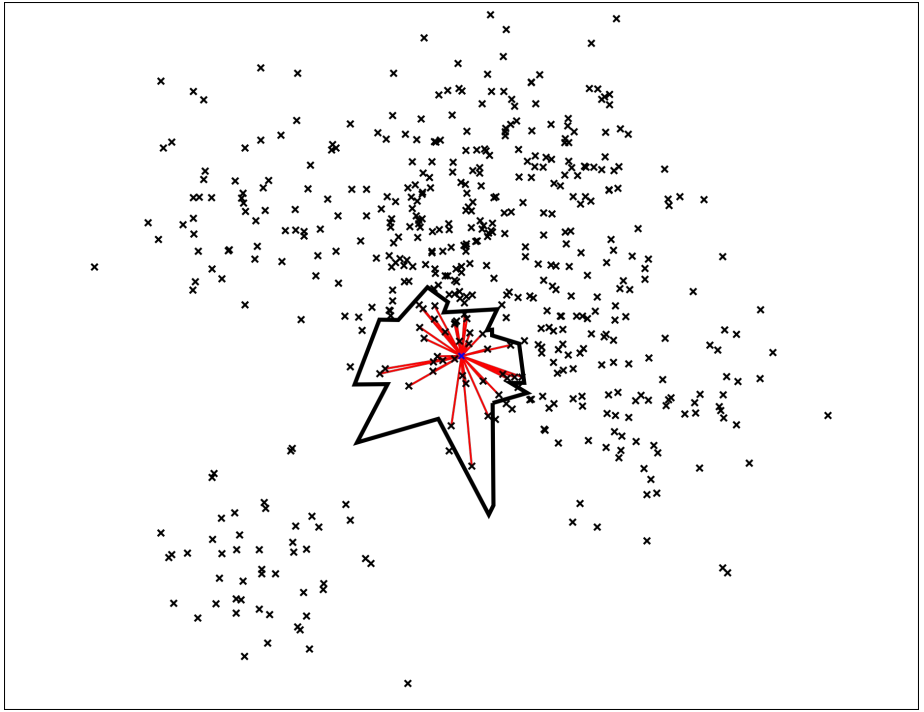
RP forest





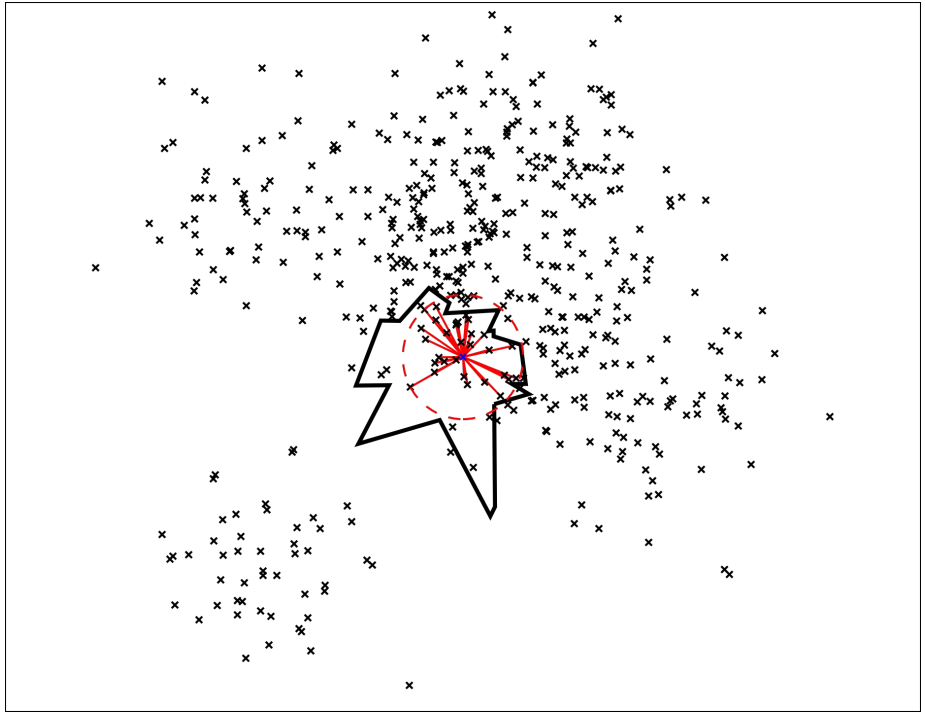
RP forest - obliczanie odległości

- brute force dla końcowego zestawu punktów
- obliczamy metrykę dla <1% punktów ze zbioru!



RP forest - hiperparametry

- nie mamy gwarancji dokładności
- **speed-recall tradeoff** wyznaczają hiperparametry:
  - liczba drzew
  - długość kolejki
- większe = dokładniej, ale dłuższy czas treningu i predykcji



RP forest - złożoność obliczeniowa

Teoria:

- F - liczba drzew
- trening: O(F*n*log(n))
- predykcja: O(F*log(n))

Praktyka:

- **bardzo szybkie**
- często wylosowane podziały są dobre -> płytkie drzewa
- bardzo mała stała
- idealnie uwspółbieżnialny

RP trees - zalety i wady

Zalety:

- prostota
- szybkość i skalowalność
- niezła dokładność
- łatwe sterowanie speed-recall tradeoff

Wady:

- dość czułe na klątwę wymiaru
- typowo wspiera mało metryk